

Reviewer Pack — Schur-Weyl Multiplicity Gap in Symmetric Powers of Permanent...

Entry 056a10374551 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-12 15:07:59 UTC

Schur-Weyl Multiplicity Gap in Symmetric Powers of Permanent vs Determinant

Entry ID: 056a10374551 Verdict: INCONCLUSIVE Recorded: 2026-05-12 15:07:59 UTC

1. Conjecture

Field A (mathematical branch): Schur-Weyl Duality **Field B** (complexity object): Boolean Circuit Size for Permanent Polynomial

Statement:

For all $n \leq 40$, the sum of squares of Kronecker coefficients in the decomposition of $S^k(\text{perm}_n)$ exceeds that of $S^k(\text{det}_m)$ by $\Omega(2^{\{n/2\}})$ for $m < n^{\{1.5\}}$, and this gap persists under linear substitutions.

Rationale (proposer’s reasoning):

Schur-Weyl duality decomposes symmetric powers into irreducible representations, revealing structural differences between permanent and determinant. The Kronecker coefficients encode combinatorial complexity, potentially linking representation-theoretic gaps to circuit lower bounds.

Taxonomy category: GCT_DET_PERM (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): ada96f7163f0781f

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.0s

5.1 Generated Python source

```
import random

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def kronecker_coefficient(a, b, c):
        if a == 0 or b == 0 or c == 0:
            return 1
        return sum(kronecker_coefficient(a-1, b-i, c) * kronecker_coefficient(i, b, c-1) for i in range(1, min(a, b, c)))

    def symmetric_power_kronecker_sum(n, k):
        total = 0
        for l in range(k+1):
            total += kronecker_coefficient(n, k-l, l) ** 2
        return total

    n = random.randint(5, 40)
    m = max(1, min(int(n ** 1.5) - 1, n - 2))
    k = (n + 1) // 2

    perm_kronecker_sum = symmetric_power_kronecker_sum(n, k)
    det_kronecker_sum = symmetric_power_kronecker_sum(m, k)

    gap = perm_kronecker_sum - det_kronecker_sum
    conjecture_holds = gap >= 2 ** (n / 2)

    return {
        "metric_name": "Kronecker Coefficient Gap",
        "metric_value": gap,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": "" if conjecture_holds else f"n={n}, m={m}"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 35)]

    results = []
```

```

for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

mean_value = sum(r["metric_value"] for r in results) / len(results)
std_value = (sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results)) ** 0.5
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
else:
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
    counterexample = next((r["counterexample"] for r in results if r["counterexample"]), "")
    print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test timed out before producing results, preventing verification of conjecture. | next:
Increase time limit and re-run for $n \leq 20$ with optimized code

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	98202
2	preregistration	ollama_remote	qwen3:8b	0	0	27181
3	novelty	ollama_remote	qwen3:8b	0	0	20736
4	novelty	ollama_remote	qwen3:8b	0	0	15572
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	44232
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7615
7	judge	ollama_remote	qwen3:8b	0	0	17097

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 230635 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in research/audit/056a10374551.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/056a10374551.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/056a10374551.tar.gz (if generated)